

CareerPilot

Your Agentic Career Co-pilot

Stack Report

Technology Justification & Architectural Decisions

Codesprint 2026 — Powered by Poridhi.io

June 8, 2026

Contents

1	Project Overview	2
2	Technology Stack	2
2.1	Frontend	2
2.2	Backend	3
2.3	Data & AI Layer	4
3	Architectural Decisions	4
3.1	RAG Over User CV as Single Source of Truth	4
3.2	Supabase as Unified Infrastructure	4
3.3	Firebase for Authentication	5
3.4	HTTP/1.1 Forced on Supabase Client	5
3.5	Dual LLM Strategy for Job Scoring vs Feed Generation	5
3.6	JSearch API with Fallback Query Strategy	5
3.7	Nudge Engine with Cooldown and Fallback	5
3.8	Groq for Cover Letter Generation	6
3.9	Email Sending via SMTP with App Password	6
3.10	Goal Completion Cascade to Tasks	6
3.11	Stateless Request Flow for Kanban and Progress	6
3.12	Scalability Considerations	6

Project Overview

CareerPilot is an end-to-end agentic career co-pilot that uses a Retrieval-Augmented Generation (RAG) architecture grounded in the user's own CV as the single source of truth. Every agent, every recommendation, and every cover letter is grounded in the specific user's actual experience — not a hallucinated generic profile.

The platform is structured around four pillars:

1. **Job Hunter Agent** — Hunts, filters, and presents job opportunities with fit scores grounded in the user's CV.
2. **Profile & Resume Intelligence (RAG Core)** — CV ingestion, chunking, embedding, and vector storage pipeline.
3. **Personal AI Assistant** — Conversational interface with full CV context for job fit, skill gap, roadmap, and cover letter generation.
4. **Productivity & Progress Tracker** — Calendar, to-do, Kanban application tracker, and progress dashboard.

Technology Stack

Frontend

Technology	Justification
Next.js 14 (App Router)	Chosen for its hybrid rendering model (SSR + CSR), file-based routing, and seamless API route support. The App Router enables React Server Components for faster initial loads.
TypeScript	Enforces type safety across the entire frontend codebase, reducing runtime errors during rapid development.
Tailwind CSS	Utility-first CSS framework enabling fast, consistent UI development without context-switching to separate stylesheets.
Firebase Auth (Client SDK)	Handles Google OAuth and session management on the client. Provides ID tokens verified server-side via Firebase Admin SDK, eliminating the need for a custom auth implementation.

Backend

Technology		Justification
FastAPI (Python)		Chosen for its async support, automatic OpenAPI documentation, and native Pydantic integration. Python was required for compatibility with the Google GenAI and firebase-admin SDKs.
Firebase SDK	Admin	Used server-side to verify Firebase ID tokens on every authenticated request, providing stateless JWT-based auth without a custom session store.
Google SDK	GenAI	Provides access to <code>gemini-embedding-2</code> for vector embedding (768 dimensions via <code>output_dimensionality</code>) and <code>gemini-3.5-flash</code> for text generation in the AI assistant and job fit scoring pipeline.
Groq SDK		Used for dashboard feed recommendation generation via <code>llama-3.1-8b-instant</code> . Chosen for its high-speed inference on a separate quota from Gemini, preventing feed generation from exhausting the Gemini rate limit.
Supabase		Serves as the unified backend-as-a-service layer: PostgreSQL for relational data, pgvector for semantic search, and object storage for raw CV files. Chosen over a separate vector database to minimize infrastructure complexity and cost.
JSearch (OpenWebNinja)	API	Provides live job listing data without the fragility of direct scraping. Used by the Job Hunter Agent with a two-pass fallback query strategy for sparse local markets.

Data & AI Layer

Technology	Justification
Supabase pgvector	PostgreSQL extension for vector similarity search. Chosen over dedicated vector databases (Pinecone, Qdrant) because embeddings and relational data share the same Supabase instance, enabling JOIN queries and eliminating cross-service network calls.
gemini-embedding-2	Google's latest embedding model. Configured with <code>output_dimensionality=768</code> to match the pgvector column definition and reduce storage compared to the default 3072 dimensions.
gemini-3.5-flash	Used for AI assistant responses and job fit scoring — produces accurate, nuanced reasoning grounded in the user's CV context.
llama-3.1-8b-instant (Groq)	Used for dashboard feed recommendations. Lighter workload handled at high speed on a separate quota, preserving Gemini rate limit for the more demanding scoring pipeline.

Architectural Decisions

RAG Over User CV as Single Source of Truth

The central architectural decision is that every AI feature is grounded in the user's actual CV via RAG, not in generic LLM knowledge. The CV ingestion pipeline follows this flow:

1. **Upload** — PDF/DOCX uploaded to Supabase Storage.
2. **Parse** — Raw text extracted via `pdfplumber` / `python-docx`.
3. **Structure** — Gemini organizes text into sections (experience, education, skills, projects).
4. **Chunk** — Sections split into semantic chunks.
5. **Embed** — Each chunk embedded via `gemini-embedding-2` at 768 dimensions.
6. **Store** — Embeddings inserted into `cv_embeddings` table with pgvector.

At query time, the user's question is embedded and the top-*k* most semantically similar CV chunks are retrieved and injected into the LLM prompt context.

Supabase as Unified Infrastructure

Rather than running ChromaDB locally (disk-heavy, crashes on low-memory machines) or managing a separate hosted vector database, all data — relational, vector, and file storage — is consolidated into a single Supabase project. This decision:

- Eliminates local disk usage for vector storage.

- Enables a single connection string and client instance.
- Allows pgvector queries to JOIN with relational tables.
- Reduces operational complexity for deployment.

Firestore for Authentication

Firestore Auth was chosen for authentication instead of Supabase Auth to leverage Google OAuth with minimal configuration. The backend verifies Firestore ID tokens via the Admin SDK on every request. User profile data is synced to Supabase's `users` table on first login via `POST /api/users/me`, after which user identity is read directly from the decoded token — no database lookup needed on every request.

HTTP/1.1 Forced on Supabase Client

The `supabase-py` client uses `httpx` under the hood, which defaults to HTTP/2. On the Supabase free tier, HTTP/2 stream multiplexing caused intermittent `ConnectionTerminated error_code:0` errors. The client is initialized with `http2=False` to force HTTP/1.1 and prevent dropped connections during embedding insertion and vector queries.

Dual LLM Strategy for Job Scoring vs Feed Generation

Job fit scoring uses `gemini-3.5-flash` while the dashboard feed recommendations use Groq's `llama-3.1-8b-instant`. This separation was intentional — Gemini produces more accurate and nuanced fit scores with detailed reasoning grounded in the user's CV, while Groq's 8b model handles the lighter feed recommendation task faster and on a completely separate quota.

This prevents feed generation from consuming the Gemini rate limit (5 requests per minute on the free tier) that is reserved for the more demanding scoring pipeline. The result is that both features remain responsive under concurrent load without competing for the same quota bucket.

JSearch API with Fallback Query Strategy

The Job Hunter uses OpenWebNinja's JSearch API rather than scraping job boards directly, avoiding the fragility and legal risk of scraping. When a query returns zero results for Bangladesh-specific searches, the agent automatically retries with a broadened query — stripped of location constraints and appended with the keyword "remote".

This two-pass strategy ensures users always see relevant results regardless of how sparse local job postings are for their role. For example, a search for "ML Engineer Dhaka" that returns zero results is automatically retried as "ML Engineer remote", surfacing globally available positions that are still actionable for the user.

Nudge Engine with Cooldown and Fallback

The nudge system uses `apscheduler` to generate daily motivational nudges, with a 20-hour cooldown to avoid spam. Generation is triggered on-demand via `POST /nudges/generate` (bypassing cooldown for manual refresh). The engine builds a user context (applications, goals, tasks) and calls `gemini-3.5-flash`. If Gemini quota is exceeded, a static fallback

message is returned based on user activity (zero applications → start applying; pending goals → focus on one; otherwise, celebrate progress). This ensures nudges are always available even under API rate limits.

Groq for Cover Letter Generation

Cover letters are generated using Groq's llama-3.3-70b-versatile rather than Gemini. This keeps the heavier Gemini quota reserved for scoring and assistant queries, while Groq offers lower latency and a separate free tier for document-length generation. The prompt enforces a strict structure (Dear Hiring Manager, interest paragraph, skills paragraph, closing) and prohibits inventing experience, ensuring professional and truthful output.

Email Sending via SMTP with App Password

The backend sends application emails via Gmail's SMTP using an App Password (not the user's regular password). This decision avoids storing user credentials and simplifies deployment. The `/kanban/send-email` endpoint requires the user's Gmail address and App Password, which is never persisted — it is used only for the duration of the request. This design is explicit about security risks and encourages users to generate a per-application token.

Goal Completion Cascade to Tasks

When a goal is marked as completed, all tasks under that goal are automatically marked as completed. This cascade behaviour is implemented in the PATCH `/calendar/goals/{goal_id}` endpoint. It ensures that progress metrics (e.g., roadmap percentage) remain consistent without requiring manual task updates. Conversely, un-completing a goal does not revert tasks, allowing users to selectively re-open tasks they have deliberately finished.

Stateless Request Flow for Kanban and Progress

All kanban, calendar, and progress endpoints are stateless: each request fetches the user's current data from Supabase, performs the update, and returns the modified result. No server-side session or cache is used. This simplifies scaling and ensures that real-time changes (e.g., drag-and-drop on the frontend) are immediately persisted and visible across devices.

Scalability Considerations

For the current hackathon scope, the architecture supports approximately 8,000 CVs within Supabase's free tier (500MB database; each CV produces approximately 20 chunks at 3KB per 768-dim vector = 60KB per CV).

For production scale (10,000+ users):

- Migrate to Supabase Pro (\$25/month) for 8GB database.
- Tune IVFFlat index `lists` parameter as vector count grows beyond 100,000.
- Introduce a job queue (Redis + Celery) for async CV ingestion to avoid blocking HTTP requests.

- Consider migrating to Qdrant Cloud when vector count exceeds 500,000.
- Upgrade Gemini to a paid tier to remove the 5 requests/minute rate limit on the scoring pipeline.